GEBZE TECHNICAL UNIVERSITY

Faculty of Sciences · Department of Data Science and Analytics

DSA 107 DATA SCIENCE AND ANALYTICS LABORATORY

LABORATORY REPORT No. 4

Getting data from the wild

Web APIs and HTML tables

Student Name	Andrei Tiniakov
Student ID	26022121102
Instructor	Salih Sarp, PhD
Lab Session	Week 4 · w/c 19 October 2026
Submission Date	25 October 2026

Gebze, Kocaeli · Academic Year 2026–2027

1. Objective

The goal of the fourth lab is to learn how to query data through an API that returns JSON, or fetch a web page that returns HTML. Both often require cleaning and normalization. We then convert the data into a DataFrame so that we can analyse it.

2. Data description

We used two datasets. We got the first through the World Bank Country API [1], and the second by querying the Wikipedia page List of countries by Human Development Index [2]. The first returns JSON, which we normalize, filter and rename to keep the fields we need. The second returns HTML, from which we select the particular table we need.

The first dataset includes rows that are aggregates of countries, which we filtered out. After filtering, we had 217 country or economy records from the original 296 records, with 6 missing longitude values and 6 missing latitude values. The other retained columns had no missing values.

The second dataset had a typical HTML problem: redundant tables. We had to find the table we needed before inspecting its elements. The 11 excluded World Bank fields are alternative ISO, region and income codes, administrative-region fields and lending classifications. They are not needed for the counts or coordinate plot. Table 1 summarizes the datasets, the columns used and the preprocessing.

Property	Value
Source	World Bank Country API [1]; Wikipedia, List of countries by Human Development Index [2].
Size and row meaning	World Bank: 296 records × 18 normalized columns, then 217 country or economy rows × 7 retained columns. HDI: 193 country or territory rows × 5 columns.
Variables and units	World Bank: region and income_level for group counts; longitude and latitude, in degrees, for capital-city locations; id, name and capitalCity for identification. HDI: Rank, Change since 2015, Country or territory, HDI value and % annual growth (2010–2023), inspected for structure and types. HDI values are index scores; annual growth is a percentage.
Missing values	World Bank: 6 missing values in each coordinate column among 217 records; none in other retained columns. HDI sample: NaN and N/A[b] markers; a full missing-value count was not calculated.
Rows used and preprocessing	World Bank: 79 aggregates removed, 7 columns kept, region.value and incomeLevel.value renamed, coordinates converted with to_numeric and errors="coerce". HDI: index 1 of 14 tables selected and inspected; no further cleaning or analysis.

Table 1. Dataset sources, structure, variable roles and preprocessing.

3. Methodology

There were five main stages in the lab. The notebook [3] uses Python with requests, pandas, seaborn and matplotlib.pyplot.

1st stage: Query the API

Here we make a request to the server URL using the requests package. We check for status code 200, meaning OK, and then read the JSON body. This gives us a Python list containing a metadata dictionary and a list of records. First, we print its contents to understand the structure. The response uses pagination, so we take the records from the second part.

2nd stage: Convert JSON into a table

We can use JSON, but it is easier for us to work with DataFrames. We use pandas.json_normalize to turn nested dictionary fields into columns, joining their names with a dot. Then we keep only the

fields we will use and rename some of them with rename. Finally, we convert longitude and latitude from strings to floating-point numbers with to_numeric, so that we can use them as quantitative coordinates.

3rd stage: Cleaning the data

Here we realize that not every row is a country, because some rows are aggregates. A boolean mask comes in handy: we keep rows where region is not "Aggregates". Then we check for missing values using isna.

4th stage: Analysis

Here we start analysing the data. First, we count how many countries or economies in each region have each income category. We use groupby on region and income_level, get the size of each group, and then unstack the result so that it is easy to read as a table. Next, we draw a figure using scatterplot. We use longitude and latitude as the axes and colour each capital-city location by region. We also place the legend outside the figure.

5th stage: Working with HTML

Here, some of the work was already provided because this is an introduction to reading HTML. We still fetch the page ourselves with a User-Agent header and use read_html to convert its tables into pandas objects. First, we find the table we need by inspecting the returned list. Then we inspect its dtypes and print the beginning and end of the table.

4. Results

At the beginning, the first dataset has 296 rows and 18 columns. One row represents a country, economy or aggregate group. Longitude and latitude are initially strings, so we convert these two columns to floating-point values. We keep seven columns and remove 79 aggregate records, leaving 217. Each coordinate column has 6 missing values. Table 2 summarizes the main outputs.

Measure or comparison	Notebook result	Units or denominator
World Bank raw table	296 × 18	Rows × normalized columns
World Bank analysis table	217 × 7	Rows × retained columns
Aggregate records removed	79	Out of 296 original records
Missing capital coordinates	6 longitude; 6 latitude	Each out of 217 retained records
Largest and smallest regions	58; 3	Europe & Central Asia; North America
HTML tables and selected HDI table	14 tables; index 1, 193 × 5	Zero-based index; rows × columns

Table 2. Main notebook results and processing checks.

Figure 1 shows the API metadata and an example country record. The response reports 296 records, matching the number received.

```
2
{'page': 1, 'pages': 1, 'per_page': '400', 'total': 296}
296
{'id': 'ABW', 'iso2Code': 'AW', 'name': 'Aruba', 'region': {'id': 'LCN', 'iso2code': 'ZJ', 'value': 'Latin America &
Caribbean '}, 'adminregion': {'id': '', 'iso2code': '', 'value': ''}, 'incomeLevel': {'id': 'HIC', 'iso2code': 'XD',
'value': 'High income'}, 'lendingType': {'id': 'LNX', 'iso2code': 'XX', 'value': 'Not classified'}, 'capitalCity':
'Oranjestad', 'longitude': '-70.0167', 'latitude': '12.5167'}
```

Figure 1. World Bank API pagination metadata and the first country record, Aruba. All 296 records were received in one page.

Europe & Central Asia has the most retained records, with 58, while North America has the fewest, with 3. Tables 3 and 4 show the grouped counts and regional totals. Some regions do not have

certain income categories at all. For instance, South Asia and high income gives NaN, meaning that there are no records in that group.

income_level	High income	Low income	Lower middle income	Upper middle income
region				
East Asia & Pacific	15.0	1.0	8.0	13.0
Europe & Central Asia	40.0	NaN	3.0	15.0
Latin America & Caribbean	19.0	NaN	5.0	18.0
Middle East, North Africa, Afghanistan & Pakistan	8.0	3.0	7.0	5.0
North America	3.0	NaN	NaN	NaN
South Asia	NaN	NaN	4.0	2.0
Sub-Saharan Africa	1.0	21.0	20.0	6.0

Table 3. Country or economy counts by World Bank region and income level, using 217 retained records. NaN marks an empty combination.

region	
East Asia & Pacific	37.0
Europe & Central Asia	58.0
Latin America & Caribbean	42.0
Middle East, North Africa, Afghanistan & Pakistan	23.0
North America	3.0
South Asia	6.0
Sub-Saharan Africa	48.0
dtype: float64	

Table 4. Total country or economy records by World Bank region. The seven totals sum to 217.

The scatter plot in Figure 2 shows capital-city locations and how they are distributed across regions. The dots look like a world map, which makes sense given their coordinates. The plot shows locations rather than country boundaries.

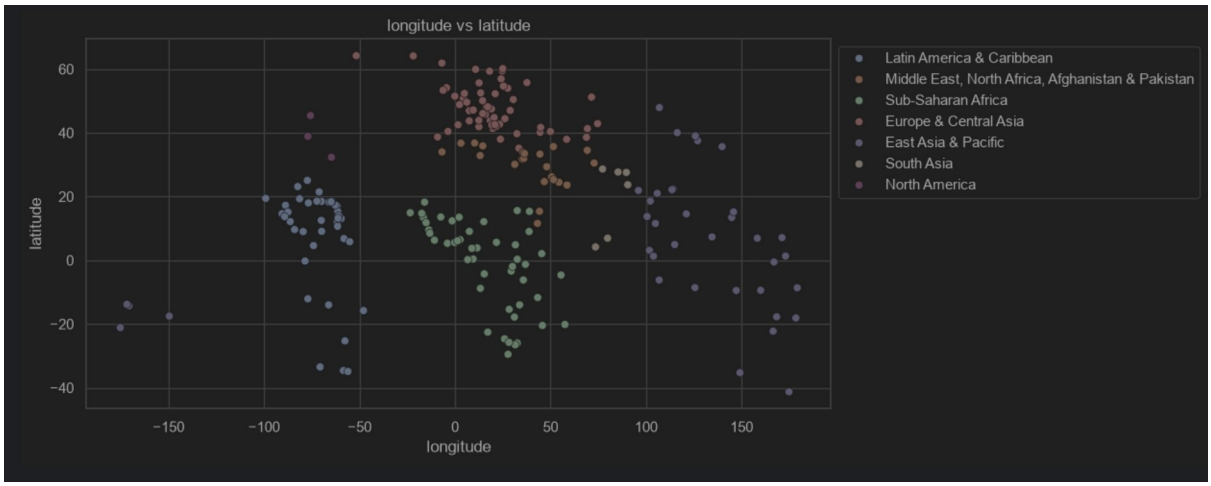


Figure 2. Capital-city longitude and latitude, in degrees, coloured by World Bank region. The source table has 217 records; rows missing either coordinate are omitted.

For the second dataset, read_html finds 14 tables. We select index 1, the second table, because it contains country HDI ranks and values. It has 193 rows and 5 columns. Figure 3 shows part of the listing of the other tables returned by read_html.

```
2 (15, 10) ['Region or group', '1990', '2000']
3 (17, 2) ['vteDeprivation and poverty indicators', 'vteDeprivation and poverty indicators.1']
4 (2, 2) [0, 1]
5 (1, 2) [0, 1]
6 (2, 2) [0, 1]
7 (2, 2) [0, 1]
8 (2, 2) [0, 1]
9 (13, 2) ['vteEconomic classification of countries', 'vteEconomic classification of countries.1']
10 (2, 2) [0, 1]
11 (13, 2) ['vteLists of countries by population statistics', 'vteLists of countries by population statistics.1']
12 (6, 2) ['vteLists of countries by quality of life rankings', 'vteLists of countries by quality of life rankings.1']
13 (8, 2) ['vteLists of subnational entities by Human Development Index rankings', 'vteLists of subnational entities by
Human Development Index rankings.1']
```

Figure 3. Partial listing of HTML tables on the Wikipedia page, showing indices 2 to 13. The selected HDI table is index 1, identified in the notebook.

The HDI annual-growth column is stored as text because its entries contain percent signs. The Change since 2015 column also contains text, parentheses and missing-value markers, so it needs cleaning before numeric conversion. Tables 5 and 6 show the column types and the first and last five rows.

Rank	int64
Change since 2015	str
Country or territory	str
HDI value	float64
% annual growth (2010–2023)	str
dtype: object	

Table 5. Data types of the five columns in the selected HDI table. Annual growth and rank changes remain stored as text.

	Rank	Change since 2015	Country or territory	HDI value	% annual growth (2010–2023)
0	1	(2)	Iceland	0.972	0.28%
1	2	(1)	Norway	0.970	0.25%
2	2	NaN	Switzerland	0.970	0.24%
3	4	(2)	Denmark	0.962	0.35%
4	5	(1)	Germany	0.959	0.19%
	Rank	Change since 2015	Country or territory	HDI value	% annual growth (2010–2023)
188	188	(3)	Niger	0.419	1.34%
189	190	(1)	Chad	0.416	0.66%
190	191	(1)	Central African Republic	0.414	0.58%
191	192	N/A[b]	Somalia	0.404	N/A[b]
192	193	NaN	South Sudan	0.388	0.61%

Table 6. First five and last five rows of the HDI table, which contains 193 countries or territories. Footnote and missing-value markers remain in the imported data.

5. Discussion

The results show how different regions contain different numbers of countries or economies in each income category. The regional totals count listed countries or economies, so differences in counts do not establish differences in income-group proportions. We can also see capital-city locations and how scattered they are. These points do not show country area or population, and rows with missing coordinates are not plotted. In the second dataset, we do not try to discover patterns; we are learning how to work with HTML properly. The web sources can change between downloads. A useful next step would be to clean the HDI percentage and rank-change columns before analysing them further.

6. Reflection

I realized that it is hard to remember the plotting API, particularly when choosing between seaborn and matplotlib.pyplot. I need to spend more time learning the difference and understanding the

benefits of each. Next time, I would keep a short example of the plotting calls and legend settings to refer to.

7. References

[1] World Bank. Country API, version 2. Accessed 25 October 2026.
https://api.worldbank.org/v2/country?format=json&per_page=400

[2] Wikipedia contributors. List of countries by Human Development Index. Accessed 25 October 2026.
https://en.wikipedia.org/wiki/List_of_countries_by_Human_Development_Index

[3] Tiniakov, A. DSA 107 Lab 4 notebook. 2026.
https://github.com/AndrewTinyakov/dsa107-labs/blob/main/04/Lab04_Andrei_Tiniakov.ipynb

Notebook repository: https://github.com/AndrewTinyakov/dsa107-labs

8. AI assistance disclosure

I used AI assistance for hints and checks on API requests, JSON data, coordinate conversion, group summaries and reading HTML tables. I used OpenAI Codex to prepare the template and lightly edit the wording, tables, captions, references and layout. The notebook conversation is included in the AI usage log at the end of the notebook.